

TECHNICAL DOCUMENTATION

ACMS Dashboard

Technical Documentation

Document	Technical Documentation
System	Access Control Management System
Organisation	NASTP
Prepared by	Muhammad Aqeel Abbas
Runtime	.NET 10 / Angular 19
Version	1.0
Date	12 August 2026

Contents

1. System Overview	3
1.1 Design constraints	3
1.2 Request flow	3
1.3 Technology stack	3
2. Prerequisites	5
3. Database Setup	6
3.1 Creating the database	6
3.2 Enabling Mixed Mode authentication	6
3.3 Identity tables	6
3.4 Verifying	6
4. Backend Configuration	8
4.1 Secrets are not in the repository	8
4.2 Non-secret configuration	8
4.3 Configuration key reference	9
5. Building and Running	10
5.1 Backend	10
5.2 Frontend	10
5.3 Ollama (offline NL agent fallback)	10
5.4 Environment files	10
6. API Surface	12
6.1 Drill-down endpoints	12
6.2 Testing endpoints manually	13
7. Subsystems	14
7.1 Real-time audit stream	14
7.2 ETL and the summary tables	14
7.3 The NL query agent	14
8. Security Model	16
8.1 Authentication	16
8.2 Authorisation	16

- 8.3 Data protection 16
 - 8.4 Rate limiting 16
 - 8.5 CORS 16
- 9. Demonstration Accounts 17
- 10. Troubleshooting 18
 - 10.1 Build and tooling 18
 - 10.2 Angular 18
 - 10.3 Runtime 18
 - 10.4 NL agent 19
- 11. Pre-Production Checklist 20
- 12. Command Reference 20

1. System Overview

The ACMS Dashboard is a two-tier web application: an ASP.NET Core Web API over the existing NASTP SQL Server access-control database, and an Angular single-page client. It adds a monitoring, analytics and workflow layer on top of that database without altering its schema.

1.1 Design constraints

- **Database-first.** The five original ACMS tables were designed by someone else and are treated as read-mostly. Entity models were scaffolded from the live schema, never migrated onto it.
- **Identity is kept separate.** ASP.NET Identity tables live in their own DbContext so that re-scaffolding the ACMS entities can never drop the user store.
- **Analytics is precomputed.** Trend charts read from three summary tables written by a scheduled job, not from live aggregation over the transactional tables.
- **Point-in-time counts are live.** “on site now” and “pending now” cannot be answered by a daily summary table, so those are queried directly and are documented as an intentional exception.

1.2 Request flow

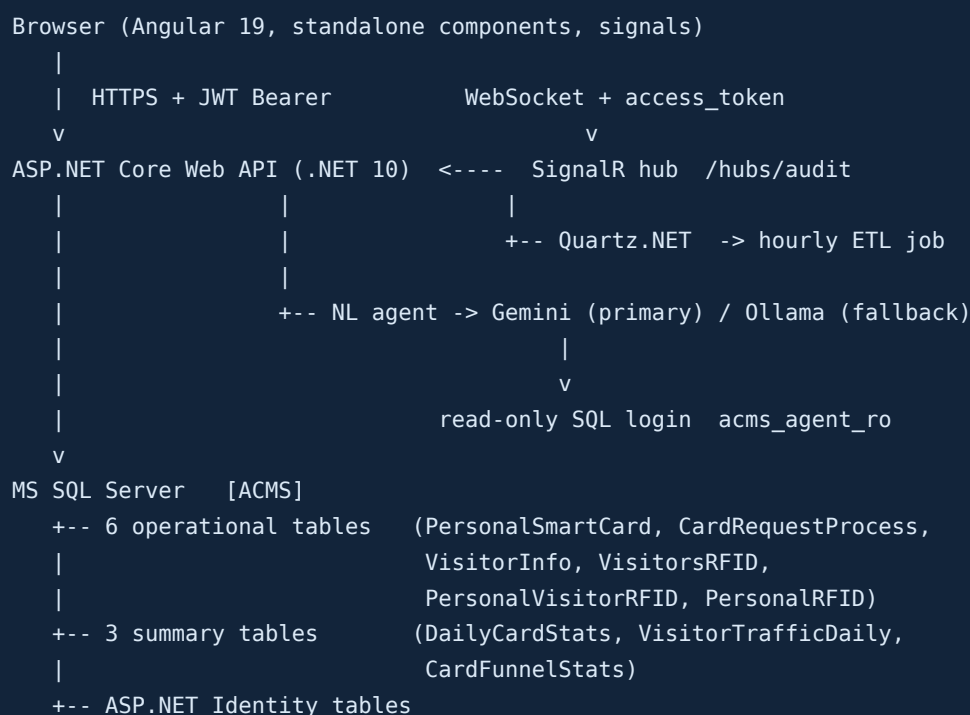


Figure 1 — component and data flow

1.3 Technology stack

Layer	Technology	Purpose
Database	MS SQL Server	System of record
ORM	Entity Framework Core 10 (database-first)	Entity mapping and LINQ queries
API	ASP.NET Core Web API, .NET 10, C#	REST endpoints
Auth	ASP.NET Identity + JWT Bearer	Accounts, roles, stateless tokens
Real time	SignalR	Audit event stream and live occupancy
Scheduling	Quartz.NET	Hourly ETL into the summary tables
Validation	FluentValidation	Request-model validation
Logging	Serilog	Structured console logging
API docs	Swashbuckle / Swagger	Interactive API explorer
NL agent	Google Gemini, with Ollama fallback	Natural-language to SQL
Frontend	Angular 19+, standalone components, signals	SPA
Charts	Chart.js	Analytics visualisations
Styling	Custom SCSS design system	Glassmorphism, light/dark theming

2. Prerequisites

Requirement	Version	Notes
Windows	10 or 11	Development target
.NET SDK	10.0 or later	Verify with <code>dotnet --version</code>
MS SQL Server	2019 or later	Express is sufficient. Mixed Mode authentication must be enabled — the NL agent needs a SQL login.
SQL Server Management Studio	19 or later	For running the setup scripts
Node.js	20 LTS or later	Required by Angular
Angular CLI	19 or later	<code>npm install -g @angular/cli</code>
Ollama	Latest	Optional but recommended — provides the offline fallback for the NL agent
Git	Any recent	Source control

Paths containing spaces

The project has historically lived at a path containing spaces (D:\FUSST BSSE\Internship\NASTP\...). Several CLI tools mishandle this. Always quote paths in shell commands, and if a tool fails for no obvious reason, try the same command from a path with no spaces before assuming the tool is broken.

3. Database Setup

Run the scripts in this exact order. Each one depends on the last.

#	Script	Creates
1	DbForDashboard.sql	The five original ACMS tables
2	PersonalRFID.sql	The staff-card table added after the first review
3	ACMS_Dummy_Data_Seed_1.sql	Demonstration data
4	Phase6_CreateSummaryTables.sql	The three analytics summary tables
5	Phase7_CreateAgentLogin.sql	The read-only SQL login used by the NL agent, with column-level DENY
6	SQLQuery4.sql	Grants the agent login SELECT on PersonalRFID

3.1 Creating the database

```
CREATE DATABASE ACMS;  
GO
```

Then open each script in SSMS in the order above and execute it against the ACMS database.

3.2 Enabling Mixed Mode authentication

The NL agent connects with a dedicated SQL login rather than the application's Windows identity, so Mixed Mode must be on.

1. In SSMS, right-click the server instance and choose **Properties**.
2. Open the **Security** page.
3. Select **SQL Server and Windows Authentication mode**.
4. Restart the SQL Server service — the change does not take effect until you do.

3.3 Identity tables

The ASP.NET Identity tables are created by an EF migration, which runs against the same database but from a separate context.

```
cd backend\AcmsDashboard.Api  
dotnet ef database update --context AppIdentityDbContext
```

3.4 Verifying

```
USE [ACMS];  
SELECT name FROM sys.tables ORDER BY name;
```

You should see the six operational tables, the three summary tables and the ASP.NET* Identity tables.

4. Backend Configuration

4.1 Secrets are not in the repository

No connection string, signing key or API key is stored in `appsettings.json`. Everything sensitive lives in .NET User Secrets, which are held outside the project folder in the developer's own profile and therefore cannot be committed by accident.

```
cd backend\AcmsDashboard.Api

dotnet user-secrets set "ConnectionStrings:AcmsDb" ^
    "Server=localhost;Database=ACMS;Trusted_Connection=True;TrustServerCertificate=True"

dotnet user-secrets set "ConnectionStrings:AgentDb" ^
    "Server=localhost;Database=ACMS;User Id=acms_agent_ro;Password=<AGENT_PASSWORD>;TrustServerCertificate=True"

dotnet user-secrets set "Jwt:Secret"    "<A LONG RANDOM STRING, 32+ CHARS>"
dotnet user-secrets set "Jwt:Issuer"    "AcmsDashboard"
dotnet user-secrets set "Jwt:Audience"  "AcmsDashboardClient"

dotnet user-secrets set "Gemini:ApiKey"  "<YOUR GEMINI API KEY>"
dotnet user-secrets set "Gemini:Model"   "gemini-3.6-flash"
```

Run from the `AcmsDashboard.Api` folder. Replace every .

Confirm they are all set:

```
dotnet user-secrets list
```

Never paste the output of that command anywhere

`dotnet user-secrets list` prints live credentials in plain text — connection strings, the JWT signing key and the Gemini API key. Do not include it in a report, a screenshot, a chat message or a commit. If a secret is ever exposed, rotate it rather than hoping it went unnoticed.

4.2 Non-secret configuration

Ollama settings are not secret and live in `appsettings.Development.json`:

```
{
  "Ollama": {
    "BaseUrl": "http://localhost:11434/",
    "Model": "llama3.1:8b",
    "TimeoutSeconds": 60
  }
}
```

4.3 Configuration key reference

Key	Where	Purpose
ConnectionStrings:AcmsDb	User Secrets	Main application connection
ConnectionStrings:AgentDb	User Secrets	Read-only connection used by the NL agent
Jwt:Secret	User Secrets	Symmetric token signing key
Jwt:Issuer	User Secrets	Expected token issuer
Jwt:Audience	User Secrets	Expected token audience
Gemini:ApiKey	User Secrets	Google AI Studio key
Gemini:Model	User Secrets	Model identifier
Ollama:BaseUrl	appsettings	Local Ollama endpoint
Ollama:Model	appsettings	Local model tag
Ollama:TimeoutSeconds	appsettings	Local generation timeout

5. Building and Running

5.1 Backend

The solution file is .slnx, not .sln

This project uses the newer XML solution format. `dotnet build` with no argument may pick up a stray file or fail to find a solution at all — always name the file explicitly.

```
cd backend
dotnet restore
dotnet build AcmsDashboard.slnx

cd AcmsDashboard.Api
dotnet run
```

The API listens on `http://localhost:5000`. Swagger is available at `http://localhost:5000/swagger` in Development only. On first run the identity seeder creates the four roles and the four demonstration accounts listed in section 9.

5.2 Frontend

```
cd frontend
npm install
ng serve
```

The client runs at `http://localhost:4200`. That exact origin is the only one allowed by the API's CORS policy, so serving the frontend from a different port will produce CORS errors until the policy is updated.

5.3 Ollama (offline NL agent fallback)

```
ollama serve
ollama pull llama3.1:8b

:: confirm it is reachable
curl http://localhost:11434/api/tags
```

The model is roughly 4.9 GB. The first generation after the service starts is slow because the model has to be loaded into memory; subsequent calls are much faster.

5.4 Environment files

File	Used by	Contains
src/environments/environment.ts	ng serve	apiUrl http://localhost:5000/v1, hubUrl http://localhost:5000/hubs
src/environments/environment.prod.ts	ng build --configuration production	Production API and hub URLs. Still placeholders — must be set before any real deployment.

6. API Surface

All routes are prefixed /v1. Every endpoint requires a valid JWT unless marked otherwise. Responses follow a single envelope: { success, data, meta } on success and { success: false, error: { code, message } } on failure.

Route group	Purpose	Role restrictions
/v1/auth	Sign in, current user	Anonymous for login
/v1/employees	Employee register	Write: Admin
/v1/card-requests	Card request workflow	Verify: Security, Admin. Print: Printer, Admin
/v1/personal-rfid	Staff cards; block and reactivate	Write: Security, Admin
/v1/visitors	Visitor log; check in and out	Write: Security, Admin
/v1/visitor-rfid	Visitor pass inventory	Write: Security, Admin
/v1/onsite	Current occupancy count	Any role
/v1/analytics	KPI summary, funnel, traffic, daily cards	ETL trigger: Admin
/v1/analytics/drilldown	Breakdown and grid for every metric	Any role
/v1/agent	Natural-language query and suggestions	Any role. Rate limited to 20/min per user
/v1/admin	User account management	Admin only
/hubs/audit	SignalR audit event stream	Token passed as access_token query parameter

6.1 Drill-down endpoints

```
GET /v1/analytics/drilldown/{kind}/summary?dimension=  
GET /v1/analytics/drilldown/{kind}/rows?category=&q=&field=&from=&to=&status=&sort=&dir=&page=  
=&limit=
```

kind is one of:

Kind	Population
total-employees	All rows in PersonalSmartCard
active-visitor-cards	VisitorsRFID where isActive = 1 and isBlocked is not 1
visitors-on-site	VisitorInfo where ExitDate is null
pending-requests	CardRequestProcess where isPrinted is not 1
active-cards	PersonalRFID where IsActive = 1
blocked-cards	PersonalRFID where IsDeactive = 1
pending-printing	CardRequestProcess, verified and not printed
pending-approval	CardRequestProcess, not verified
visitors-today	VisitorInfo where EntryDate is today (UTC)

Every metric publishes its own definition

The summary response carries a definition field containing the exact SQL predicate behind the total, and the UI surfaces it under **How this is counted**. This exists because a review found the dashboard and SSMS reporting different counts for the same concept with no way to see which filter each had applied. When a figure is questioned, that field is the answer.

6.2 Testing endpoints manually

Swagger's **Authorize** button has known issues attaching the bearer token on this Swashbuckle version. PowerShell with an explicit header is the reliable route:

```
$login = Invoke-RestMethod -Uri http://localhost:5000/v1/auth/login `
    -Method Post -ContentType 'application/json' `
    -Body '{"username":"admin","password":"Admin@12345"}'

$token = $login.data.token

Invoke-RestMethod -Uri http://localhost:5000/v1/analytics/summary `
    -Headers @{ Authorization = "Bearer $token" }
```

7. Subsystems

7.1 Real-time audit stream

Controllers raise audit events through `IAuditBroadcaster`, which pushes them over the SignalR hub at `/hubs/audit`. The browser cannot set an `Authorization` header on a WebSocket handshake, so the JWT is passed as an `access_token` query parameter and picked up in `JwtBearerEvents.OnMessageReceived` for paths under `/hubs`.

Reconnection behaviour

The client uses an unlimited exponential-backoff retry policy capped at 30 seconds, plus a manual recovery path after a hard close and a `visibilitychange` listener that reconnects when the tab regains focus. A fixed retry array was tried first and gave up permanently after roughly 37 seconds, so any backend restart longer than that silently stopped all live updates until the user signed out and back in.

7.2 ETL and the summary tables

`DailyStatsJob` runs hourly under Quartz.NET and calls `EtlService.RunAsync`, which backfills 90 days by default. Each stage is attributed to the date it occurred on: submitted to `ProcessDate`, verified to `MarkedOn`, printed to `PrintingDate`. Writes are an upsert keyed on the date, so re-running is idempotent and never duplicates rows.

Administrators can trigger it on demand: `POST /v1/analytics/etl/run?lookbackDays=90`.

Event dates and cohorts are different questions

The daily bars are an event series — each stage counted on the day it happened. The funnel totals are a cohort — requests submitted inside the window, followed to whatever stage they have now reached. Summing the daily columns and dividing printed by submitted mixes the two and can exceed 100%, which is exactly what happened before the funnel totals were moved onto a cohort basis. Keep the distinction if this code is extended.

7.3 The NL query agent

A question travels through five stages:

1. The question, plus a schema description, goes to the language model, which returns a SQL `SELECT` statement.
2. `SqlSafetyValidator` rejects anything that is not a single plain `SELECT` over an allow-listed table.
3. The statement runs on the `acms_agent_ro` connection, which has `SELECT` only and column-level `DENY` on biometric and document columns.
4. The result set goes back to the model for a plain-English summary.
5. The answer is returned. The generated SQL is deliberately not sent to the client.

Provider failover

FailoverNlQueryClient tries Gemini first and falls back to Ollama on any failure — quota exhaustion, an auth error, a timeout, or no network at all. The failover is logged server-side and is never surfaced to the caller: the user simply gets an answer. Note that each question makes **two** model calls (SQL generation, then summarisation), so provider timeouts apply twice per question.

Defence in depth

Layer	Enforced by	Blocks
1 — Database	SQL login permissions	Any write; any read of biometric, document or address columns. This is the real boundary.
2 — Validator	SqlSafetyValidator	Non-SELECT statements, multiple statements, comments, SELECT *, system objects, tables outside the allow list
3 — Transport	AgentController	The generated SQL never reaches the browser
4 — Rate limit	ASP.NET rate limiter	20 agent requests per minute per user

Never weaken layer 1 on the strength of layer 2

The validator exists to reject obviously bad queries early with a clear message. It is not the security boundary. The SQL login's GRANT and DENY set is, and it must stay at least as restrictive as it is today.

8. Security Model

8.1 Authentication

- ASP.NET Identity with EF Core storage, in its own DbContext.
- Passwords: minimum 8 characters, at least one uppercase, one digit, one symbol. Five failed attempts lock the account.
- JWT bearer tokens, symmetrically signed, validating issuer, audience, lifetime and signing key with a one-minute clock skew.
- Sign-in checks both the Identity logout state and the application's own IsActive flag. These are separate mechanisms and both must agree, or deactivation is cosmetic only.

8.2 Authorisation

Four roles: Admin, Security, Printer, Viewer. Role checks are declared on controller actions with `[Authorize(Roles = "...")]`, and the Angular client mirrors them for UI visibility only. The client-side checks are convenience, not security — every restriction is enforced server-side as well.

8.3 Data protection

These columns are denied to the agent login at the database level and are never selected by any dashboard endpoint:

Table	Denied columns
PersonalSmartCard	Picture, QRCode, FPEenroll, CNICBlob, FIRBlob, PoliceVerificationBlob, AttachmentBlob, PresentAddress, PermanentAddress
VisitorInfo	Picture
AspNetUsers	Not in the agent's allow list at all

8.4 Rate limiting

Scope	Limit	Window
Global, per user or IP	120 requests	1 minute
NL agent, per user or IP	20 requests	1 minute

Exceeding either returns HTTP 429 with error code `RATE_LIMITED`.

8.5 CORS

The API allows exactly one origin, `http://localhost:4200`. This must be changed before the application is served from anywhere else.

9. Demonstration Accounts

Development only — remove before any production use

These four accounts are created automatically on first run by `IdentitySeeder.cs`, and their credentials are also shown as one-click chips on the login screen in development builds. Both the seeding and the chips must be removed or gated before the application is exposed to anyone outside the project. See section 11.

Username	Password	Role	Seeded as
admin	Admin@12345	Admin	Muhammad Aqeel Abbas
security1	Security@12345	Security	Hassan Raza
printer1	Printer@12345	Printer	Bilal Ahmed
viewer1	Viewer@12345	Viewer	Ayesha Khan

The read-only SQL login `acms_agent_ro` is created by `Phase7_CreateAgentLogin.sql`. Its password is set inside that script and is a development value; change it there and in the `ConnectionStrings:AgentDb` secret before any real use.

10. Troubleshooting

Every entry below is a problem that actually occurred during development, with the resolution that worked.

10.1 Build and tooling

VS Code shows dozens of errors but `dotnet build` succeeds.

Trust the build. The C# language server reports phantom errors when a stray or empty `.slnx` file exists in the wrong folder. Delete it, then reload the window. Press **Ctrl+K S** to save all files before every build — unsaved buffers are the other common cause.

CS0246: a type that clearly exists is not found.

The file most likely contains invisible non-ASCII characters introduced by repeated copy-paste. Do not try to find them. Delete the file entirely, create it again, and paste once.

A newly created file appears correct but the build behaves as if it is empty.

Files created through VS Code's New File and then pasted into have been observed saving empty or partially written. Delete and recreate rather than editing in place.

`dotnet build` cannot find a solution.

The solution file is `AcmsDashboard.slnx`, not `.sln`. Run `dotnet build AcmsDashboard.slnx` from the backend folder.

The API crashes immediately on startup.

Check that `<StaticWebAssetsEnabled>false</StaticWebAssetsEnabled>` is still present in the API `.csproj`. Without it the SDK's static web assets bootstrap can fail hard at startup.

10.2 Angular

The Angular language service reports errors that no longer exist.

Delete the `.angular/cache` folder and restart VS Code completely. A window reload is not always enough.

A component's blur or glass effect silently does nothing.

A nested `backdrop-filter` inside a parent that already applies one is dropped by the browser. Use a solid opaque background for the inner element instead — this is why the notification panel and the export menu are opaque by design.

A CSS custom property appears to be ignored.

It probably does not exist. Undefined custom properties fail silently to the browser default. Check the name against `styles.scss`.

10.3 Runtime

Times display several hours out.

EF Core returns SQL Server datetime values with `Kind=Unspecified`, and `System.Text.Json` then serialises them without a trailing Z, so the browser treats a UTC value as local. Timestamps must be marked as UTC explicitly before serialisation, or have a Z appended client-side.

Swagger's Authorize button does not attach the token.

A known incompatibility on this OpenApi/Swashbuckle combination. Use the PowerShell approach in section 6.2 instead.

Live updates stop after the backend restarts.

This was fixed by the unlimited-backoff retry policy described in section 7.1. If it recurs, check the browser console for hub connection errors and confirm the token has not expired.

CORS errors in the browser console.

The API allows only `http://localhost:4200`. Confirm the frontend is on that exact port.

10.4 NL agent

A reasonable question is blocked for security reasons.

The generated SQL referenced a table missing from `SqlSafetyValidator.AllowedTables`, or one the agent login has no GRANT on, or a column matching the sensitive-column pattern. Adding a table requires three changes together: the allow list, the schema description in `AgentService.SchemaContext`, and a `GRANT SELECT` to `acms_agent_ro`. Missing any one of the three produces this symptom — that is exactly what happened when `PersonalRFID` was first added.

The agent reports a network error when the machine is offline.

Each question makes two model calls, and each tries Gemini before Ollama. Offline that is up to two 30-second Gemini timeouts plus two Ollama calls, and a cold Ollama model load can exceed its own 60-second budget. Raise `Ollama:TimeoutSeconds`, keep `ollama serve` running so the model stays warm, and see the roadmap item on short-circuiting Gemini when it is known to be unreachable.

The answer is cut off, or the SQL is invalid partway through.

The model hit its output token ceiling and truncated mid-statement rather than failing cleanly. `maxOutputTokens` is set to 2048 for this reason; compound UNION-style questions can still approach it.

Gemini stops answering after about twenty-five questions in a day.

That is the free tier's requests-per-day limit. The Ollama fallback takes over automatically, which is precisely what it is there for.

11. Pre-Production Checklist

Every item below is a development convenience that must be dealt with before the application is used by anyone outside the project team.

#	Item	Location	Action
1	Demo credential chips	login.component.ts	Gate behind <code>!environment.production</code>
2	Four seeded accounts	IdentitySeeder.cs	Gate behind a <code>seedDemoUsers</code> flag; always seed roles
3	JWT signing secret	User Secrets	Replace with a fresh 32+ character random value
4	Production API URLs	environment.prod.ts	Replace the placeholders
5	Gemini API key	User Secrets	Issue a key scoped to this application
6	Database connection strings	User Secrets	Point at the production instance
7	acms_agent_ro password	Phase7_CreateAgentLogin.sql	Change in the script and in the AgentDb secret
8	CORS origin	Program.cs	Replace <code>localhost:4200</code> with the real origin, config-driven
9	Security headers	SecurityHeadersMiddleware	Prepared but not yet registered — apply
10	HTTPS and HSTS	Program.cs	Confirm enabled outside Development
11	Swagger	Program.cs	Already Development-only — confirm it stays that way

12. Command Reference

Task	Command
Restore packages	<code>dotnet restore</code>
Build backend	<code>dotnet build AcmsDashboard.slnx</code>
Run API	<code>dotnet run (from AcmsDashboard.Api)</code>
Apply identity migration	<code>dotnet ef database update --context AppIdentityDbContext</code>
List secrets	<code>dotnet user-secrets list</code>
Set a secret	<code>dotnet user-secrets set "Key" "Value"</code>
Install frontend packages	<code>npm install</code>
Run frontend	<code>ng serve</code>
Production frontend build	<code>ng build --configuration production</code>
Start Ollama	<code>ollama serve</code>
Pull the local model	<code>ollama pull llama3.1:8b</code>
Check Ollama is up	<code>curl http://localhost:11434/api/tags</code>
Clear Angular cache	<code>rmdir /s /q .angular\cache</code>
Save all files in VS Code	<code>Ctrl+K S</code>

The single most useful habit

`dotnet build` is the authoritative check. Editor squiggles on this project have been wrong repeatedly — stray solution files, invisible characters and stale language-server caches have all produced errors that did not exist. Save everything, build, and believe the build.